6

机制：受限直接执行

为了虚拟化（virtualize）CPU，操作系统需要设法让许多看起来同时运行的作业共享一个物理 CPU。基本思路很简单：让一个进程（process）运行一小会儿，再让另一个进程运行一小会儿，如此轮流下去。通过这种方式**分时共享**（time sharing）CPU，就实现了虚拟化。

不过，构建这样的虚拟化机制有几个难点。第一个是性能：怎样实现虚拟化，才不会给系统增加过多的开销？第二个是控制：怎样才能在高效运行进程的同时，保持对 CPU 的控制？控制对操作系统尤其重要，因为它负责管理资源；没有控制，进程就可能一直运行下去、占据整台机器，或者访问它不该访问的信息。因此，在保持控制的同时获得高性能，是构建操作系统的核心挑战之一。

关键问题：
如何在保持控制的同时高效地虚拟化 CPU
操作系统必须高效地虚拟化 CPU，同时保持对系统的控制。为此，既需要硬件支持，也需要操作系统支持。操作系统常常会审慎地借助一点硬件支持，以便有效地完成工作。

6.1 基本技术：受限直接执行

不出所料，为了让程序跑得像人们期望的那样快，操作系统的开发者想出了一种技术，我们把它叫作**受限直接执行**（limited direct execution）。这个想法中“直接执行”的部分很简单：直接在 CPU 上运行程序就行了。因此，当操作系统想启动一个程序时，它会在进程列表（process list）中为这个程序创建一个进程条目，为它分配一些内存，（从磁盘）把程序代码加载到内存中，找到程序的入口点（即 `main()` 例程或类似的东西），跳转过去，开始运行用户的代码。图 6.1 展示了这个基本的直接执行协议（暂时还没有任何限制），它用普通的调用和返回跳转到程序的 `main()`，之后再回到内核。

听起来很简单，不是吗？但在虚拟化 CPU 的过程中，这种方法会带来几个问题。第一个问题很简单：如果直接运行程序，操作系统怎样在高效运行程序的同时，确保程序不做我们不希望它做的事？第二个问题：进程运行时，操作系统怎样让它停下来、切换到另一个进程，从而实现虚拟化 CPU 所需的**分时共享**（time sharing）？

操作系统	程序
在进程列表中创建条目	
为程序分配内存	
把程序加载到内存中	
用 argc/argv 设置栈	
清除寄存器	
执行**调用** main()	
	运行 main()
	从 main 执行**返回**
释放进程的内存	
把进程从进程列表中删除	

图 6.1：**直接执行协议（无限制）**

在下面回答这些问题的过程中，我们会更清楚地了解虚拟化 CPU 需要什么。在设计这些技术时，我们还会看到名字里“受限”这一部分从何而来；如果对运行的程序不加限制，操作系统就什么都控制不了，因而只能算“一个库”——对一个有抱负的操作系统来说，这可真是件可悲的事！

6.2 问题 1：受限的操作

直接执行有一个明显的好处：快。程序直接在硬件 CPU 上运行，因此执行速度和人们预期的一样快。但在 CPU 上直接运行会带来一个问题：如果进程想执行某种受限的操作，比如向磁盘发出 I/O 请求，或者获取更多的系统资源（比如 CPU 或内存），该怎么办?

关键问题：如何执行受限的操作
进程必须能够执行 I/O 和其他一些受限的操作，但又不能让进程完全控制整个系统。操作系统和硬件怎样协作才能做到这一点?

一种办法是干脆让任何进程在 I/O 和其他相关操作上为所欲为。然而，这样一来，许多我们想要的系统就无法构建了。例如，如果我们想构建一个在允许访问文件之前检查权限的文件系统（file system），就不能让任何用户进程都可以直接向磁盘发出 I/O；否则，进程完全可以读写整个磁盘，所有保护也就形同虚设了。

因此，我们采用的方法是引入一种新的处理器模式，叫作**用户模式**（user mode）；在用户模式下运行的代码，能做的事情受到限制。例如，在用户模式下运行时，进程不能发出 I/O 请求；如果它这样做，处理器就会触发异常，然后操作系统多半会杀掉这个进程。

与用户模式相对的是**内核模式**（kernel mode），操作系统（即内核）就运行在这种模式下。在内核模式下，运行的代码可以为所欲为，包括执行特权操作（privileged operation），比如发出 I/O 请求和执行各种受限的指令。

不过，我们还剩下一个难题：当用户进程想执行某种特权操作时，比如从磁盘读数据，它该怎么办? 为此，几乎所有现代硬件都为用户程序提供了执行**系统调用**（system call）的

 © 2008–23, ARPACI-DUSSEAU

补充：为什么系统调用看起来像过程调用

你可能会好奇，为什么调用 `open()` 或 `read()` 这样的系统调用（system call），看起来和 C 语言中普通的过程调用（procedure call）一模一样；也就是说，既然看起来就像过程调用，系统怎么知道它是系统调用，并做好该做的一切？原因很简单：它就是一个过程调用，只不过在这个过程调用里藏着著名的陷阱（trap）指令。具体来说，当你调用（比如说）`open()` 时，你执行的是一个进入 C 库的过程调用。在库里，无论是 `open()` 还是其他任何系统调用，库都会遵循与内核商定的调用约定，把 `open()` 的参数放到约定好的位置（比如栈上，或者特定的寄存器里），把系统调用号（system-call number）也放到约定好的位置（同样是栈上或寄存器里），然后执行前面说的陷阱（trap）指令。陷阱（trap）之后，库中的代码会取出返回值，把控制权交还给发出系统调用的程序。因此，C 库中负责发出系统调用的那部分代码是用汇编手写的，因为它们必须严格遵守约定，才能正确处理参数和返回值，并执行与具体硬件相关的陷阱（trap）指令。现在你知道了，为什么你自己不必写汇编代码来陷入操作系统：已经有人替你写好了。

能力。系统调用最早出现在 Atlas [K+61, L78] 这类古老的机器上，它让内核可以小心地把某些关键功能开放给用户程序，比如访问文件系统、创建和销毁进程、与其他进程通信，以及分配更多内存。大多数操作系统提供几百个调用（详见 POSIX 标准 [P10]）；早期的 UNIX 系统则只开放了一个更精简的子集，大约二十个调用。

要执行系统调用，程序必须执行一条特殊的**陷阱**（trap）指令。这条指令会跳转到内核，同时把特权级别提升到内核模式；一旦进入内核，系统就可以执行所需的任何特权操作（如果允许的话），替调用进程完成所需的工作。完成后，操作系统调用一条特殊的**从陷阱返回**（return-from-trap）指令；你可能已经猜到，它会返回到发起调用的用户程序，同时把特权级别降回用户模式。

硬件在执行陷阱（trap）时需要格外小心，因为它必须保存足够多的调用者寄存器，以便操作系统发出从陷阱返回（return-from-trap）指令时能正确返回。例如，在 x86 上，处理器会把程序计数器（program counter）、标志位和另外几个寄存器压入该进程的**内核栈**（kernel stack，每个进程各有一个）；从陷阱返回（return-from-trap）时，再把这些值从栈中弹出，恢复执行用户模式程序（详见 Intel 的系统手册 [I11]）。其他硬件系统使用的约定不同，但基本概念在各个平台上都差不多。

提示：使用受保护的控制权转移

硬件通过提供不同的执行模式来帮助操作系统。在**用户模式**（user mode）下，应用程序不能完全访问硬件资源。在**内核模式**（kernel mode）下，操作系统可以访问机器的全部资源。硬件还提供了专门的指令：**陷入**（trap）内核的指令、**从陷阱返回**（return-from-trap）用户模式程序的指令，以及让操作系统告诉硬件**陷阱表**（trap table）在内存中什么位置的指令。

上面的讨论漏掉了一个重要细节：陷阱（trap）怎么知道在操作系统内部运行哪段代码？显然，调用进程不能像过程调用那样指定要跳转到的地址；否则，程序就能跳到内核里的任何地方，这显然是个**非常糟糕的主意**[1]。因此，内核必须严格控制陷阱（trap）发生时执行哪些代码。

内核的做法是在启动时设置**陷阱表**（trap table）。机器启动时处于特权（内核）模式，因此可以根据需要自由地配置机器硬件。所以，操作系统最早做的事情之一，就是告诉硬件在发生某些异常事件时运行哪段代码。例如，发生硬盘中断（interrupt）、发生键盘中断，或者程序发出系统调用时，应该运行哪段代码？操作系统通常用某种特殊指令，把这些**陷阱处理程序**（trap handlers）的位置告诉硬件。硬件得到通知后，会记住这些处理程序的位置，直到机器下次重启；这样，在发生系统调用和其他异常事件时，硬件就知道该怎么做（即跳转到哪段代码）。

为了指定具体是哪个系统调用，通常会给每个系统调用分配一个**系统调用号**（system-call number）。因此，用户代码负责把所需的系统调用号（system-call number）放到寄存器中，或者栈上的指定位置；操作系统在陷阱处理程序（trap handler）中处理系统调用时，会检查这个号码，确认它有效，如果有效，就执行相应的代码。这种间接层（level of indirection）起到了一种**保护**（protection）作用；用户代码不能指定确切的跳转地址，而必须通过号码来请求某项特定的服务。

最后再说一句：能够执行“告诉硬件陷阱表（trap table）在哪里”的指令，是一种非常强大的能力。所以你可能已经猜到，它也是一种**特权**（privileged）操作。如果你试图在用户模式下执行这条指令，硬件不会允许，而且你大概能猜到会发生什么（提示：再见了，违规的程序）。思考题：如果你能安装自己的陷阱表（trap table），你能对系统干出哪些可怕的事？你能接管整台机器吗？

图 6.2 中的时间线（时间自上而下递增）总结了这个协议（protocol）。我们假设每个进程都有一个内核栈，在进入和退出内核时，（由硬件）把寄存器（包括通用寄存器和程序计数器，program counter）保存到这里，再从这里恢复。

受限直接执行（**LDE**）协议分为两个阶段。在第一阶段（启动时），内核初始化陷阱表（trap table），CPU 记住陷阱表的位置供以后使用。内核通过一条特权指令（privileged instruction）完成这项工作（所有特权指令都用粗体标出）。在第二阶段（运行进程时），内核先做好一些准备（比如在进程列表中分配一个节点、分配内存），然后用一条从陷阱返回（return-from-trap）指令开始执行进程；这会把 CPU 切换到用户模式，并开始运行进程。当进程想发出系统调用时，它会陷入操作系统，由操作系统处理，然后再次通过从陷阱返回（return-from-trap）把控制权交还给进程。之后进程完成自己的工作，从 `main()` 返回；这通常会返回到一段存根代码中，由它正确地退出程序（比如调用 `exit()` 系统调用，这会陷入操作系统）。这时，操作系统完成清理工作，整个过程就结束了。

[1]想象一下，跳进一段访问文件的代码，而且恰好跳过了权限检查；事实上，有了这种能力，狡猾的程序员很可能就能让内核运行任意的代码序列 [S07]。总之，尽量避免这类非常糟糕的主意。

操作系统 @ 启动 **（内核模式）**	**硬件**	
初始化陷阱表		
	记住以下处理程序的地址…… 系统调用处理程序	
操作系统 @ 运行 **（内核模式）**	**硬件**	**程序** **（用户模式）**
在进程列表中创建条目		
为程序分配内存		
把程序加载到内存中		
用 argv 设置用户栈		
在内核栈中填入寄存器/PC		
从陷阱返回		
	（从内核栈）恢复寄存器 切换到用户模式 跳转到 main	
		运行 main() ……
		调用系统调用 **陷入**操作系统
	（向内核栈）保存寄存器 切换到内核模式 跳转到陷阱处理程序	
处理陷阱 完成系统调用的工作		
从陷阱返回		
	（从内核栈）恢复寄存器 切换到用户模式 跳转到陷阱之后的 PC	
		……
		从 main 返回 **陷入**（通过 `exit()`）
释放进程的内存 把进程从进程列表中删除		

图 6.2：**受限直接执行协议**

6.3 问题 2：在进程之间切换

直接执行的下一个问题是如何在进程之间切换。在进程之间切换应该很简单，对吧？操作系统只要决定停下一个进程、启动另一个进程就行了。这有什么大不了的？但实际上这有点棘手：具体来说，如果一个进程正在 CPU 上运行，那么按照定义，操作系统就没有在运行。如果操作系统没有在运行，它又怎么能做任何事呢？（提示：它做不到）这听起来几乎像个哲学问题，但它是个实实在在的问题：如果操作系统没有在 CPU 上运行，它显然无法采取任何行动。于是，我们就来到了问题的关键。

提示：在安全的系统中，要提防用户的输入
尽管我们在系统调用期间费了很大力气来保护操作系统——加入了硬件陷阱（trap）机制，并确保对操作系统的所有调用都经过它——实现一个**安全的**（secure）操作系统还有许多其他方面需要考虑。其中之一是处理系统调用边界上的参数；操作系统必须检查用户传入的内容，确保参数指定得正确，否则就拒绝这次调用。
例如，对于 `write()` 系统调用，用户要指定一个缓冲区（buffer）的地址，作为写入的数据来源。如果用户（无论是无意还是恶意）传入了一个“错误的”地址（比如位于地址空间（address space）中内核那一部分的地址），操作系统必须发现这一点并拒绝这次调用。否则，用户就有可能读取内核的全部内存；而内核的（虚拟）内存通常还包含系统的全部物理内存（physical memory），所以这个小小的疏忽，就能让程序读取系统中任何其他进程的内存。
总之，安全的系统必须对用户输入抱有高度的怀疑。不这样做，无疑会导致软件很容易被攻破，让人绝望地觉得世界是个不安全的可怕地方，还会让过于轻信的操作系统开发者丢掉饭碗。

关键问题：如何重新获得 CPU 的控制权
操作系统怎样才能**重新获得** CPU 的**控制权**（regain control），以便在进程之间切换？

协作方式：等待系统调用

过去一些系统采用的一种方法（例如早期版本的 Macintosh 操作系统 [M11]，以及老式的 Xerox Alto 系统 [A79]）叫作**协作**（cooperative）方式。在这种方式下，操作系统相信系统中的进程会表现得合乎情理。操作系统假定，运行时间太长的进程会定期让出（yield）CPU，好让操作系统决定去运行别的任务。

于是你可能会问：在这个理想世界里，友好的进程怎么让出（yield）CPU 呢？事实证明，大多数进程会相当频繁地通过发出**系统调用**（system call）把 CPU 的控制权交给操作系统，比如打开文件然后读取它、向另一台机器发送消息，或者创建新进程。这类系统通常还会提供一个显式的 **yield** 系统调用，它什么都不做，只是把控制权交给操作系统，好让操作系统运行其他进程。

应用程序做了非法的事情时，也会把控制权交给操作系统。例如，如果应用程序除以零，或者试图访问它不该访问的内存，就会产生一个**陷阱**（trap），陷入操作系统。这时操作系统又重新控制了 CPU，并且多半会终止这个违规的进程。

因此，在协作式调度系统中，操作系统靠等待系统调用或某种非法操作的发生，来重新获得 CPU 的控制权。你可能也在想：这种被动的方法是不是不太理想？比如，如果某个进程陷入了死循环（无论它是恶意的，还是满是 bug），再也不发出系统调用，会怎么样？那时操作系统又能怎么办？

非协作方式：操作系统接管控制权

事实证明，如果没有硬件的额外帮助，当进程拒绝发出系统调用（或者出错）、从而不把控制权交还给操作系统时，操作系统几乎什么也做不了。事实上，在协作方式（cooperative approach）下，进程陷入死循环时，你唯一的办法就是求助于计算机系统中解决一切问题的古老方案：**重启机器**。于是，我们又遇到了“获得 CPU 控制权”这个总目标下的一个子问题。

关键问题：如何在没有协作的情况下获得控制权
即使进程不配合，操作系统怎样才能获得 CPU 的控制权？操作系统能做些什么，确保失控的进程不会接管整台机器？

答案其实很简单，而且很多年前就被许多构建计算机系统的人发现了：**时钟中断**（timer interrupt）[M+63]。可以对定时器设备编程，让它每隔若干毫秒产生一次中断；中断产生时，当前运行的进程会被暂停，操作系统中预先配置好的**中断处理程序**（interrupt handler）开始运行。此时，操作系统重新获得了 CPU 的控制权，于是可以随心所欲：停止当前进程，启动另一个进程。

和前面讨论系统调用时一样，操作系统必须告诉硬件，发生时钟中断时运行哪段代码；因此，操作系统在启动时就做了这件事。另外，同样是在启动过程中，操作系统必须启动定时器，这当然是一项特权操作。定时器一旦启动，操作系统就可以放心，因为控制权最终总会回到它手里，于是它可以自由地运行用户程序。定时器也可以关闭，这同样是一项特权操作；等我们更深入地了解并发（concurrency）之后，再讨论这一点。

注意，发生中断时，硬件也要承担一些责任，尤其是要为中断发生时正在运行的程序保存足够多的状态，以便随后的从陷阱返回（return-from-trap）指令能够正确地恢复这个程序的运行。这一系列动作和硬件在显式的系统调用陷入内核时的行为非常相似：各种寄存器被保存下来（比如保存到内核栈上），因此从陷阱返回（return-from-trap）指令可以很容易地恢复它们。

保存和恢复上下文

既然操作系统已经重新获得了控制权，无论是通过系统调用协作地获得，还是通过时钟中断更强硬地夺回，接下来都要做一个决定：是继续运行当前正在运行的进程，还是切换到

提示：处理应用程序的不当行为
操作系统常常要处理行为不端的进程，也就是那些出于故意（恶意）或者意外（bug）而试图做不该做的事的进程。在现代系统中，操作系统处理这类不当行为的办法，就是直接终止违规者。一次出局！也许有些残酷，但当你试图非法访问内存或执行非法指令时，操作系统还能怎么办呢？

提示：用时钟中断重新获得控制权
加入**时钟中断**（timer interrupt）之后，即使进程以不配合的方式运行，操作系统也能重新在 CPU 上运行。因此，这项硬件特性对于帮助操作系统保持对机器的控制至关重要。

另一个进程。这个决定由操作系统中一个叫作**调度程序**（scheduler）的部分做出；接下来的几章会详细讨论调度策略（scheduling policy）。

如果决定切换，操作系统就会执行一段底层代码，我们称之为**上下文切换**（context switch）。上下文切换在概念上很简单：操作系统只要为当前正在执行的进程保存几个寄存器的值（比如保存到它的内核栈上），再为即将执行的进程恢复几个寄存器的值（从它的内核栈中恢复）。这样，操作系统就能确保，当最终执行从陷阱返回（return-from-trap）指令时，系统不会回到原来那个正在运行的进程，而是恢复执行另一个进程。

为了保存当前正在运行的进程的上下文，操作系统会执行一些底层汇编代码，保存当前进程的通用寄存器、PC 和内核栈指针，然后恢复即将执行的进程的这些寄存器和 PC，并切换到它的内核栈。通过切换栈，内核在被中断的那个进程的上下文中进入对切换代码的调用，却在即将执行的那个进程的上下文中返回。当操作系统最终执行从陷阱返回（return-from-trap）指令时，即将执行的进程就成了当前正在运行的进程。上下文切换就此完成。

图 6.3 展示了整个过程的时间线。在这个例子中，进程 A 正在运行，然后被时钟中断打断。硬件把 A 的寄存器保存到 A 的内核栈上，然后进入内核，切换到内核模式。在时钟中断处理程序（timer interrupt handler）中，操作系统决定从运行进程 A 切换到运行进程 B。这时，它调用 `switch()` 例程，这个例程小心地保存当前的寄存器值（保存到 A 的进程结构（process structure）中），恢复进程 B 的寄存器（从 B 的进程结构条目中），然后**切换上下文**（switches contexts），具体做法是修改栈指针（stack pointer），改用 B 的内核栈（而不是 A 的）。最后，操作系统从陷阱返回（return-from-trap），恢复 B 的寄存器，并开始运行 B。

提示：重启很有用
前面提到，在协作式抢占（cooperative preemption）下，死循环（以及类似行为）唯一的解决办法就是重启机器。你可能会对这种拙劣的办法嗤之以鼻，但研究人员已经证明，重启（或者更一般地说，让某个软件从头再来）在构建健壮的系统时可能是一个极其有用的工具 [C+04]。
具体来说，重启之所以有用，是因为它把软件带回到一个已知的、很可能经过了更多测试的状态。重启还会回收过时的或泄漏的资源（比如内存），这些资源用别的办法可能很难处理。最后，重启很容易自动化。出于上述种种原因，在大规模集群的互联网服务中，系统管理软件定期**重启**（reboot）一批机器、让它们复位，从而获得上面列出的好处，并不是什么稀罕事。
所以，下次你重启机器时，你可不只是在用某种拙劣的办法应付了事，而是在用一种久经考验的方法改善计算机系统的行为。干得好！

操作系统 @ 启动（内核模式）	**硬件**
初始化陷阱表	
	记住以下处理程序的地址…… 系统调用处理程序 定时器处理程序
启动中断定时器	
	启动定时器 X 毫秒后中断 CPU

操作系统 @ 运行（内核模式）	**硬件**	**程序（用户模式）**
		进程 A ……
	时钟中断 保存 regs(A) → k-stack(A) 切换到内核模式 跳转到陷阱处理程序	
处理陷阱 调用 `switch()` 例程 保存 regs(A) → proc_t(A) 恢复 regs(B) ← proc_t(B) 切换到 k-stack(B) **从陷阱返回（进入 B）**		
	恢复 regs(B) ← k-stack(B) 切换到用户模式 跳转到 B 的 PC	
		进程 B ……

图 6.3：**受限直接执行协议（时钟中断）**

注意，在这个协议中发生了两种寄存器的保存和恢复。第一种发生在时钟中断时；这时，正在运行的进程的*用户寄存器*由*硬件*隐式地保存，用的是这个进程的内核栈。第二种发生在操作系统决定从 A 切换到 B 时；这时，*内核寄存器*由*软件*（即操作系统）显式地保存，不过这次是保存到内存中这个进程的进程结构（process structure）里。后一个动作让系统从“好像刚从 A 陷入内核”变成“好像刚从 B 陷入内核”。

为了让你更清楚地了解这种切换是怎么实现的，图 6.4 给出了 xv6 的上下文切换代码。看看你能不能看懂它（要看懂，你得懂一点 x86，还得懂一点 xv6）。上下文结构 `old` 和 `new` 分别位于旧进程和新进程的进程结构（process structure）中。

```
1  # void swtch(struct context *old, struct context *new);
2  #
3  # 把当前的寄存器上下文保存到 old 中，
4  # 然后从 new 中加载寄存器上下文。
5  .globl swtch
6  swtch:
7    # 保存旧的寄存器
8    movl 4(%esp), %eax  # 把 old 指针放进 eax
9    popl 0(%eax)        # 保存旧的 IP
10   movl %esp, 4(%eax)  # 以及栈指针
11   movl %ebx, 8(%eax)  # 以及其他寄存器
12   movl %ecx, 12(%eax)
13   movl %edx, 16(%eax)
14   movl %esi, 20(%eax)
15   movl %edi, 24(%eax)
16   movl %ebp, 28(%eax)
17
18   # 加载新的寄存器
19   movl 4(%esp), %eax  # 把 new 指针放进 eax
20   movl 28(%eax), %ebp # 恢复其他寄存器
21   movl 24(%eax), %edi
22   movl 20(%eax), %esi
23   movl 16(%eax), %edx
24   movl 12(%eax), %ecx
25   movl 8(%eax), %ebx
26   movl 4(%eax), %esp  # 栈在这里切换
27   pushl 0(%eax)       # 把返回地址放到位
28   ret                 # 最后返回到新的上下文中
```

图 6.4：**xv6 的上下文切换代码**

6.4 担心并发?

你们中一些专心又爱思考的读者，现在可能在想：“嗯……如果在系统调用期间发生了时钟中断，会怎么样?”或者“如果正在处理一个中断时又来了另一个中断，会怎么样? 内核处理起来不会很难吗?”问得好——看来你们还是很有希望的!

答案是肯定的，操作系统确实需要考虑在处理中断或陷阱（trap）期间又发生另一个中断时会怎样。事实上，这正是本书整个第二部分讨论的主题，也就是**并发**（concurrency）；我们会把详细的讨论推迟到那时。

为了吊吊你的胃口，我们先简单勾勒一下操作系统处理这些棘手情况的一些基本思路。操作系统可以做的一件简单的事，是在处理中断期间**禁用中断**（disable interrupts）；这样可以确保在处理一个中断时，不会有其他中断被送到 CPU。当然，操作系统这样做时必须小心；中断禁用得太久，可能导致中断丢失，而这（用技术术语来说）很糟糕。

操作系统还开发了许多精巧的**加锁**（locking）方案，来保护对内部数据结构（data structure）的并发访问。这样，内核中就可以同时进行多项活动，这在多处理器

补充：上下文切换要花多长时间

你自然会问：上下文切换这样的操作要花多长时间？系统调用呢？对此好奇的读者可以用一个叫作 **lmbench** [MS96] 的工具，它测量的正是这些东西，还有其他几项可能相关的性能指标。

随着时间的推移，测量结果改善了很多，大致和处理器性能的提升同步。例如，1996 年在 200 MHz 的 P6 CPU 上运行 Linux 1.3.37 时，一次系统调用大约需要 4 微秒，一次上下文切换大约需要 6 微秒 [MS96]。现代系统的表现几乎好了一个数量级，在 2 GHz 或 3 GHz 处理器的系统上，结果都在 1 微秒以下。

需要指出的是，并非所有的操作系统操作都和 CPU 性能同步提升。正如 Ousterhout 所观察到的，许多操作系统操作是内存密集型的，而这些年来内存带宽（bandwidth）的提升远不如处理器速度那么显著 [O90]。因此，取决于你的工作负载（workload），买最新最好的处理器，未必能像你希望的那样让操作系统变快。

（multiprocessor）上尤其有用。不过，我们会在本书关于并发的下一部分中看到，这类加锁可能很复杂，还会导致各种有趣又难以发现的 bug。

6.5 小结

我们介绍了实现 CPU 虚拟化的一些关键底层机制，我们把这组技术统称为**受限直接执行**（limited direct execution）。基本思路很直接：直接在 CPU 上运行你想运行的程序，但事先要设置好硬件，限制进程在没有操作系统协助的情况下能做的事。

现实生活中也有类似的做法。例如，有孩子的人，或者至少听说过孩子的人，可能熟悉给房间做**婴儿防护**（baby proofing），让婴儿可以安全活动：把装着危险物品的柜子锁上，把电源插座盖住。房间这样准备好之后，你就可以放心地让孩子随意走动，因为你知道房间里最危险的地方都已经被限制住了。

与此类似，操作系统也给 CPU“做了婴儿防护（baby proofing）”：先（在启动时）设置好陷阱处理程序（trap handler）、启动中断定时器（interrupt timer），然后只在受限的模式下运行进程。这样，操作系统就可以相当放心：进程能够高效地运行，只有在执行特权操作，或者独占 CPU 太久、需要被换下时，才需要操作系统介入。

至此，我们有了虚拟化 CPU 的基本机制。但还有一个重大问题没有回答：在某一时刻应该运行哪个进程？回答这个问题正是调度程序的任务，也是我们接下来要研究的主题。

补充：CPU 虚拟化关键术语（机制）

- CPU 应该至少支持两种执行模式：受限的**用户模式**（user mode）和特权的（不受限的）**内核模式**（kernel mode）。
- 典型的用户应用程序在用户模式下运行，通过**系统调用**（system call）**陷入**（trap）内核，请求操作系统提供服务。
- 陷阱（trap）指令会小心地保存寄存器状态，把硬件状态切换到内核模式，然后跳转到操作系统中预先指定的目的地：**陷阱表**（trap table）。
- 操作系统处理完系统调用后，通过另一条特殊的**从陷阱返回**（return-from-trap）指令返回到用户程序；这条指令会降低特权级别，并把控制权交还给跳入操作系统的那条陷阱（trap）指令之后的指令。
- 陷阱表（trap table）必须由操作系统在启动时设置好，并确保用户程序无法轻易修改它。所有这些都属于**受限直接执行**（limited direct execution）协议的一部分，这个协议既能高效地运行程序，又不会让操作系统失去控制。
- 程序一旦开始运行，操作系统就必须借助硬件机制，确保用户程序不会永远运行下去，这个机制就是**时钟中断**（timer interrupt）。这种方法是 CPU 调度的一种**非协作**（non-cooperative）方式。
- 有时，操作系统在时钟中断或系统调用期间，可能想从运行当前进程切换到运行另一个进程，这种底层技术叫作**上下文切换**（context switch）。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[A79] "Alto User's Handbook" by Xerox. Xerox Palo Alto Research Center, September 1979. Available: http://history-computer.com/Library/AltoUsersHandbook.pdf. *An amazing system, way ahead of its time. Became famous because Steve Jobs visited, took notes, and built Lisa and eventually Mac.*

[C+04] "Microreboot – A Technique for Cheap Recovery" by G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, A. Fox. OSDI '04, San Francisco, CA, December 2004. *An excellent paper pointing out how far one can go with reboot in building more robust systems.*

[I11] "Intel 64 and IA-32 Architectures Software Developer's Manual" by Volume 3A and 3B: System Programming Guide. Intel Corporation, January 2011. *This is just a boring manual, but sometimes those are useful.*

[K+61] "One-Level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Transactions on Electronic Computers, April 1962. *The Atlas pioneered much of what you see in modern systems. However, this paper is not the best one to read. If you were to only read one, you might try the historical perspective below [L78].*

[L78] "The Manchester Mark I and Atlas: A Historical Perspective" by S. H. Lavington. Communications of the ACM, 21:1, January 1978. *A history of the early development of computers and the pioneering efforts of Atlas.*

[M+63] "A Time-Sharing Debugging System for a Small Computer" by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), May, 1963, New York, USA. *An early paper about time-sharing that refers to using a timer interrupt; the quote that discusses it: "The basic task of the channel 17 clock routine is to decide whether to remove the current user from core and if so to decide which user program to swap in as he goes out."*

[MS96] "lmbench: Portable tools for performance analysis" by Larry McVoy and Carl Staelin. USENIX Annual Technical Conference, January 1996. *A fun paper about how to measure a number of different things about your OS and its performance. Download lmbench and give it a try.*

[M11] "Mac OS 9" by Apple Computer, Inc.. January 2011. Available at the following URL: http://en.wikipedia.org/wiki/Mac_OS_9. *You can probably even find an OS 9 emulator out there if you want to; check it out, it's a fun little Mac!*

[O90] "Why Aren't Operating Systems Getting Faster as Fast as Hardware?" by J. Ousterhout. USENIX Summer Conference, June 1990. *A classic paper on the nature of operating system performance.*

[P10] "The Single UNIX Specification, Version 3" by The Open Group, May 2010. Available: http://www.unix.org/version3/. *This is hard and painful to read, so probably avoid it if you can. Like, unless someone is paying you to read it. Or, you're just so curious you can't help it!*

[S07] "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)" by Hovav Shacham. CCS '07, October 2007. *One of those awesome, mind-blowing ideas that you'll see in research from time to time. The author shows that if you can jump into code arbitrarily, you can essentially stitch together any code sequence you like (given a large code base); read the paper for the details. The technique makes it even harder to defend against malicious attacks, alas.*

作业（测量）

补充：测量作业
测量作业是一些小练习：你要编写代码，在真实的机器上运行，测量操作系统或硬件性能的某个方面。这类作业的目的，是让你对真实的操作系统有一点亲手操作的经验。

在这个作业中，你要测量系统调用和上下文切换的开销。测量系统调用的开销相对容易。例如，你可以反复调用一个简单的系统调用（比如读取 0 字节），并测量所花的时间；用总时间除以迭代次数，就能估算出一次系统调用的开销。

你必须考虑的一件事，是计时器的精度和准确度。你可以用的一个典型计时器是 `gettimeofday()`；详情请读手册页（manual page）。你会在手册页中看到，`gettimeofday()` 返回自 1970 年以来的时间，以微秒为单位；但这并不意味着这个计时器能精确到微秒。连续调用 `gettimeofday()` 并测量结果，了解这个计时器到底有多精确；这会告诉你，空系统调用（null system call）测试要迭代多少次，才能得到可靠的测量结果。如果 `gettimeofday()` 对你来说不够精确，可以研究一下 x86 机器上的 `rdtsc` 指令。

测量上下文切换的开销要稍微棘手一些。lmbench 基准测试的做法是：在单个 CPU 上运行两个进程，并在它们之间建立两个 UNIX 管道（pipe）；管道只是 UNIX 系统中进程之间相互通信的众多方式之一。第一个进程向第一个管道写入数据，然后等待从第二个管道读取数据；操作系统看到第一个进程在等待从第二个管道读数据，就把它置于阻塞状态，并切换到另一个进程；另一个进程从第一个管道读取数据，然后向第二个管道写入数据。当第二个进程再次试图从第一个管道读数据时，它也会阻塞，于是这种来回通信的循环就持续下去。通过反复测量这种通信的开销，lmbench 就能较好地估算出上下文切换的开销。你可以用管道，或者其他通信机制（比如 UNIX 套接字（socket）），尝试实现类似的测量。

在多 CPU 系统上测量上下文切换的开销会遇到一个困难：你必须确保进行上下文切换的进程位于同一个处理器上。幸运的是，大多数操作系统都提供了把进程绑定到某个处理器上的调用；例如，在 Linux 上，你要找的就是 `sched_setaffinity()` 调用。确保两个进程位于同一个处理器上，你测量的就是操作系统在同一个 CPU 上停下一个进程、恢复另一个进程的开销。

 © 2008–23, ARPACI-DUSSEAU